

Examen réparti 2

Master 1 SAR

UE MU4IN401

L. Arantes, P. Sens, J. Sopena,

Janvier 2022

1h30 – **Aucun document autorisé**

Tout appareil électronique interdit

Le barème est donné à titre indicatif

Exercice 1 – Système de fichiers (5 points)

On considère le fichier "./fich" qui contient uniquement la chaîne de caractères "examen2022" et est désigné par l'inode 30. Les inodes ont la même structure que celle vue en TD. Un programme fait les instructions suivantes :

```
1 int main() {  
2     int fd1, fd2;  
3     char buf[4];  
4     fd1 = open("./fich", O_RDONLY);  
5     fd2 = open("./fich", O_RDWR);  
6     write(fd2, "BBB", 3);  
7     if (fork() == 0) { 1 fil  
8         if (fork() == 0) { 2 fils  
9             close(fd2);  
10            read(fd1, buf, 3);  
11            buf[3] = 0;  
12            printf("1- %s\n", buf);  
13            exit(0);  
14        }  
15        read(fd2, buf, 3);  
16        buf[3] = 0;  
17        printf("2- %s\n", buf);  
18        wait(NULL);  
19        exit(0);  
20    }  
21    wait(NULL);  
22    return 0;  
23 }
```

Question 1 1 point

Quels sont les affichages faits par ce programme et par quels processus ?

Question 2 1/2 point

Quel est le contenu du fichier "./fich" à la fin de l'exécution ?

Question 3 1½ points

Donnez l'état, juste après la ligne 10 et en supposant que la ligne 15 a été exécutée, des tables de descripteurs de fichiers (*u_ofile*), fichiers ouverts (*file*), et inodes (*inode*). Faites un schéma en pensant à représenter les champs *f_count*, *f_offset*, *f_flag*, *f_finode*, *i_count*, *i_number* et *i_size*. Pour les inodes vous représenterez juste l'état de l'inode 30.

On considère la partition numéro 0 avec des blocs de 512 octets. Le superbloc correspond au bloc 1. La table des inodes commence à partir du bloc 2 inclus. Les inodes sont numérotées à partir de 1. Les inodes ont une taille de 32 octets. Le répertoire courant est désigné par l'inode 10. Nous considérons les inodes et les blocs suivants :

i_number = 10		i_number = 30	
i_addr[]		i_addr[]	
0	101	0	500
1	102	1	
2	103	2	
3	104	3	
4	105	4	
5	106	5	
6	107	6	
7	108	7	
8	109	8	
9	110	9	
10	300	10	
11	-	11	-
12	-	12	-

Bloc 300	Bloc 230	Bloc 500
230	« fich » 30	examen2022

Question 4 ½ point

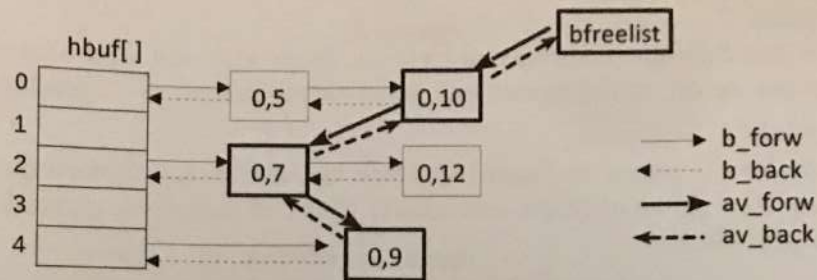
Dans quels blocs sont situés sur disque les inodes 10 et 30 ? Donnez la formule qui, à partir d'un numéro d'inode, retourne le numéro de bloc sur disque.

Question 5 1½ points

On suppose que seule l'inode 10 (le répertoire courant) est initialement en **mémoire**. Quels sont, dans l'ordre, les blocs et inodes accédés par les lignes 4 et 6 de ce programme ? Quel est le nombre d'entrées/sorties en supposant qu'aucun des blocs n'est initialement en mémoire et que le buffer cache peut contenir tous les blocs ?

Exercice 2 – Buffer cache (5 points)

Nous considérons un buffer cache de 5 cases. La figure suivante donne l'état du buffer cache. Dans chaque buffer est indiqué le doublet $\langle dev, bloc \rangle$ qui correspond au numéro de device et de bloc associés au buffer.



Question 1 $\frac{1}{2}$ point

Dans cette configuration, quelle est la fonction de hachage $fh(dev, bloc)$ qui permet de répartir les blocs dans hbuf?

Question 2 2 points

On suppose qu'aucun buffer n'a initialement le flag DEL_WRI. La séquence suivante est exécutée :

```

1 struct buf *bp;
2 bp = bread(0,16);
3 brelse(bp);
4 bp=bread(0,10);
5 brelse(bp);
6 bp = bdwrite(0,9);

```

Donnez l'état de la bfreelist (la liste des blocs qui la compose de la tête vers la queue) après les lignes 3, 5 et 6 ainsi que le nombre d'entrées/sorties physiques générées par cette séquence (les codes de getblk, brelse, bread et bdwrite sont rappelés en annexe).

Question 3 1 point

Faites un schéma complet du buffer cache (hbuf, chainage des buffers et de la bfreelist) après la ligne 6.

Question 4 $1\frac{1}{2}$ points

On remplace le code par la séquence suivante :

```

1 struct buf *bp;
2 bp = breada(0,16,10);
3 brelse(bp);
4 bp = bdwrite(0,9);

```

A quel moment et par quelle fonction, le buffer contenant le bloc $\langle 0,10 \rangle$ sera inséré dans la bfreelist par un appel à brelse? Quel est maintenant l'état de la bfreelist après la ligne 3 en considérant que la fonction insérant $\langle 0,10 \rangle$ dans bfreelist n'a pas été encore appelée?

Exercice 3 – Appel système - Fichier (5 points)

On souhaite implémenter dans le noyau du TD l'appel système `int checkaccess (int fd)`. Cet appel vérifie si le processus courant est le seul parmi l'ensemble des processus du système à utiliser l'inode du fichier désigné par `fd`, `fd` étant le descripteur retourné par l'appel système `open` (i.e., `fd = open(...)`). La fonction retourne 0 si le processus est seul et 1 sinon. Pour simplifier, on supposera que les processus ne peuvent pas appeler les appels système `dup` ou `dup2` et que `fd` désigne bien un fichier préalablement ouvert.

Question 1 $1\frac{1}{2}$ points

Il est possible qu'un même processus utilise plusieurs fois un même fichier (par exemple s'il a ouvert plusieurs fois le même fichier sans faire de `close`). Programmez une fonction interne `int countused(struct inode *ip)` qui retourne le nombre de fois que le processus courant utilise le fichier d'inode `ip`.

Question 2 $\frac{1}{2}$ point

Pourquoi est-il possible que `int countused(struct inode *ip)` soit égal à `ip->i_count` alors que le fichier désigné par `ip` est utilisé également par d'autres processus? Justifiez votre réponse.

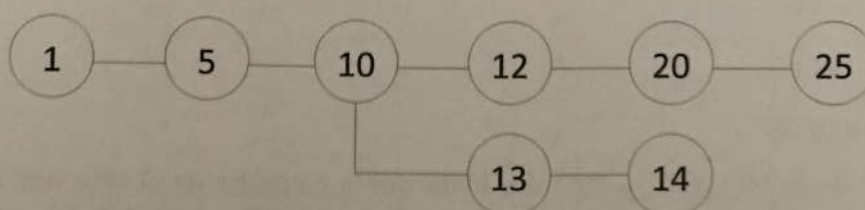
Question 3 3 points

Donnez le code dans le noyau de l'appel système `sys_checkaccess()` correspondant à `int checkaccess (int fd)` en utilisant `countused()` (que l'on supposera correct) et en tenant compte de la question précédente.

Exercice 4 – Appel système - Processus (5 points)

Nous voulons offrir l'appel système `int common_ancestor (int id)` qui retourne le pid de l'ancêtre commun le plus proche entre le processus appelant et le processus de pid `id`.

Par exemple si on considère l'arbre de processus suivant dans lequel sont représentés les pid et les liens de filiation :



Si le processus 14 appelle `common_ancestor(25)`, la valeur 10 lui sera retournée.

Question 1 $\frac{1}{2}$ point

Est-il possible que deux processus dont le pid est supérieur à 1 n'aient pas d'ancêtre commun? Justifiez votre réponse.

Question 2 $1\frac{1}{2}$ points

Donnez le code d'une fonction interne au noyau `struct proc *find_father(struct proc *p)` qui retourne un pointeur sur la structure `proc` du père du processus `p` en supposant `p->p_pid > 1`.

Question 3 3 points

Donnez le code dans le noyau de l'appel système `sys_common_ancestor()` correspondant à `int common_ancestor (int id)`. Si le processus `id` n'existe pas l'appel doit retourner l'erreur `ECHILD`. Indication : vous pouvez utiliser la fonction `struct proc *find_father(struct proc)` définie à la question précédente.

Annexe

```
#define NADDR 13

struct inode
{
    char          i_flag;
    char          i_count; /* reference count */
    dev_t         i_dev;   /* device where inode resides */
    ino_t         i_number; /* i number, 1-to-1 with device address */
    unsigned short i_mode;
    short         i_nlink; /* directory entries */
    short         i_uid;   /* owner */
    short         i_gid;   /* group of owner */
    off_t         i_size;  /* size of file */
    struct {
        daddr_t   i_addr[NADDR]; /* if normal file/directory */
        daddr_t   i_lastr;       /* last logical block read (for read-ahead) */
    };
};

struct inode inode[NINODE]; /* The inode table itself */

/*
 * One file structure is allocated
 * for each open/creat/pipe call.
 */
struct file
{
    char    f_flag;
    cnt_t   f_count; /* reference count */
    int     *f_inode; /* pointer to inode structure */
    long     f_offset; /* read/write character index */
} file[NFILE];

/* flags */
#define FOPEN    (-1)
#define FREAD    0x0001
#define FWRITE    0x0002
#define FPIPE    0x0004
```

```

struct proc
{
    short    p_addr;      /* address of swappable image */
    short    p_size;      /* size of swappable image (in blocks) */
    int      p_flag;      /* process flags */
    char     p_stat;      /* process state */
    char     p_pri;       /* priority, negative is high */
    char     p_nice;      /* nice for scheduling */
    long     p_sig;       /* signal number sent to this process */
    short    p_uid;       /* real user id, used to direct tty signals */
    short    p_suid;      /* set (effective) user id */
    short    p_time;      /* resident time for scheduling */
    int      p_cpu;       /* cpu usage for scheduling */
    short    *p_ttyp;     /* controlling tty */
    short    p_pid;       /* unique process id */
    short    p_ppid;      /* process id of parent */
    caddr_t  p_wchan;     /* event process is awaiting */
    struct text *p_textp; /* pointer to text structure */
    short    p_tsize;     /* size of text */
    short    p_ssize;     /* size of stack */
    short    p_clktim;    /* time to alarm clock signal */
} proc[NPROC];

```

```

struct user
{
    int      u_rsav[2];   /* saved env. for process switching */
    int      u_ssav[2];   /* saved env. for swapping */
    int      u_qsav[2];   /* saved env. for signaling */
    struct proc *u_procp; /* pointer to proc structure */
    char     u_error;     /* return error code */
    char     u_intflg;     /* catch intr from sys */
    int      *u_ar0;       /* address of users saved R0 */
    int      u_arg[20];    /* arguments to current system call */
    int      *u_ap;        /* pointer to arglist */
    struct file *u_ofile[NOFILE]; /* pointers to open file */
    int      u_signal[NSIG]; /* disposition of signals */
    int      u_uid;        /* effective user id */
    int      u_gid;        /* effective group id */
    int      u_ruid;       /* real user id */
    int      u_rgid;       /* real group id */
    int      u_tsize;      /* text size (in blocs) */
    int      u_dsize;      /* data size (in blocs) */
    int      u_ssize;      /* stack size (in blocs) */
    int      u_csize;      /* amount of stack in use (in blocs) */
    ...
} u;

```

```

/*
 * Read in (if necessary) the block and return a buffer pointer.
 */
struct buf *
bread(dev, blkno)
    dev_t dev;
    daddr_t blkno;
{
    register struct buf *bp;

    bp = getblk(dev, blkno);
    if (bp->b_flags & B_DONE)
        return(bp);
    bp->b_flags |= B_READ;
    bp->b_bcount = BSIZE;
    (*bdevsw[bmajor(dev)].d_strategy)(bp);
    iowait(bp);
    return(bp);
}

/*
 * Read in the block, like bread, but also start I/O on the
 * read-ahead block (which is not allocated to the caller)
 */
struct buf *
breada(dev, blkno, rablkno)
    dev_t dev;
    daddr_t blkno, rablkno;
{
    register struct buf *bp, *rabp;

    bp = NULL;
    if (!incore(dev, blkno)) {
        bp = getblk(dev, blkno);
        if ((bp->b_flags & B_DONE) == 0) {
            bp->b_flags |= B_READ;
            bp->b_bcount = BSIZE;
            (*bdevsw[bmajor(dev)].d_strategy)(bp);
        }
    }
    if (rablkno && bfreelist.b_bcount > 1 && !incore(dev, rablkno)) {
        rabp = getblk(dev, rablkno);
        if (rabp->b_flags & B_DONE)
            brelse(rabp);
        else {
            rabp->b_flags |= B_READ | B_ASYNC;
            rabp->b_bcount = BSIZE;
            (*bdevsw[bmajor(dev)].d_strategy)(rabp);
        }
    }
    if (bp == NULL)
        return(bread(dev, blkno));
    iowait(bp);
    return(bp);
}

```



```

/* Release the buffer, marking it so that if it is grabbed
 * for another purpose it will be written out before being
 * given up (e.g. when writing a partial block where it is
 * assumed that another write for the same block will soon follow).
 * This can't be done for magtape, since writes must be done
 * in the same order as requested.
 */
bdwrite(bp)
register struct buf *bp;
{
    bp->b_flags |= B_DELWRI | B_DONE;
    bp->b_resid = 0;
    brelse(bp);
}

/*
 * release the buffer, with no I/O implied.
 */
brelse(bp)
register struct buf *bp;
{
    register struct buf **backp;
    register s;

    if (bp->b_flags & B_WANTED)
        wakeup((caddr_t)bp);
    if (bfreelist.b_flags & B_WANTED) {
        bfreelist.b_flags &= ~B_WANTED;
        wakeup((caddr_t)&bfreelist);
    }
    if (bp->b_flags & B_ERROR) {
        bp->b_flags &= ~(B_ERROR | B_DELWRI);
        bp->b_error = 0;
    }

    s = gpl();
    spl(BDINHB);
    backp = &bfreelist.av_back;
    (*backp)->av_forw = bp;
    bp->av_back = *backp;
    *backp = bp;
    bp->av_forw = &bfreelist;
    bp->b_flags &= ~(B_WANTED | B_BUSY | B_ASYNC);
    bfreelist.b_bcount++;
    spl(s);
}

/*
 * Assign a buffer for the given block. If the appropriate
 * block is already associated, return it; otherwise search
 * for the oldest non-busy buffer and reassign it.
 */
struct buf *

```



```

getblk(dev, blkno)
    register dev_t dev;
    daddr_t blkno;
{
    register struct buf *bp;
    register struct buf *dp;

loop:
    spl(NORMAL);
    dp = bhash(dev, blkno);
    if (dp == NULL)
        panic("devtab");
    for (bp=dp->b_forw; bp != dp; bp = bp->b_forw) {
        if (bp->b_blkno!=blkno || bp->b_dev!=dev)
            continue;
        spl(BDINHB);
        if (bp->b_flags&B_BUSY) {
            bp->b_flags |= B_WANTED;
            sleep((caddr_t)bp, PRIBIO+1);
            goto loop;
        }
        spl(NORMAL);
        notavail(bp);
        return(bp);
    }
    spl(BDINHB);
    if (bfreelist.av_forw == &bfreelist) {
        bfreelist.b_flags |= B_WANTED;
        sleep((caddr_t)&bfreelist, PRIBIO+1);
        goto loop;
    }
    spl(NORMAL);
    bp = bfreelist.av_forw;
    notavail(bp);
    if (bp->b_flags & B_DELWRI) {
        bp->b_flags |= B_ASYNC;
        bwrite(bp);
        goto loop;
    }
    bp->b_flags = B_BUSY;
    bp->b_back->b_forw = bp->b_forw;
    bp->b_forw->b_back = bp->b_back;
    bp->b_forw = dp->b_forw;
    bp->b_back = dp;
    dp->b_forw->b_back = bp;
    dp->b_forw = bp;
    bp->b_dev = dev;
    bp->b_blkno = blkno;
    return(bp);
}

/*
 * Wait for I/O completion on the buffer; return errors
 * to the user.
 */
iowait(bp)
    register struct buf *bp;
{

```

```

spl(BDINHB);
while ((bp->b_flags&B_DONE)==0)
    sleep((caddr_t)bp, PRIBIO);
spl(NORMAL);
geterror(bp);
}

```

```

iodone(bp)
register buf *bp
{
    bp->b_flags |= B_DONE;
    if((bp->b_flags&B_ASYNC)!=0)
        brelse(bp);
    else
        wakeup((caddr_t)bp);
}

```